

Adding a global contract assumption mode

Abstract

This paper proposes adding a global contract assumption mode, in order to avoid undesired optimization-assumptions of unchecked contracts. This allows safer adoption of contract checking, when it's not necessary to make sure contract checks are 100% correct to the point that they can be assumed to be true. The solution has been shamelessly stolen from private communications with Bjarne; any good parts of it are his, all the bugs are mine. This paper should hopefully be redundant with P1711's minimal-change alternative; it's published separately just to make it abundantly certain that the approach isn't left unproposed.

Rationale

See P1711 for more analysis and rationale. My personal dulcet tones follow. It's worth mentioning that I do not support removing continuation mode; it has its uses, and I haven't been convinced that its potential abuses or other dangers justify removing it.

In C++20 contracts as we have them in the working paper, checking and assumptions are coupled. If you want one, you get the other. It has been suggested to be plausible by multiple audiences that the assumption part is highly undesirable. This is especially the case when the level of confidence for the contracts being correctly expressed is not high, or if there's a new library/application combination that hasn't been tried before. In such cases and many others, it's beneficial to be able to turn contract checking off, but also turn assumptions off.

Previous attempts to solve this problem have failed; plausibly because many of them tried to remove functionality that existed in the WP. This proposal doesn't do that; everything that is possible to express in the current WP is still possible to express, and more, because now we can have "checking=off;assumption=off" so we will provide a richer feature set that (feel free to consider this 'allegedly') enables adopting (and using) contracts in more use cases.

Why this is important, and cannot wait until C++23

I need to be able to write contracts without worrying about whether they are 100% correct from the point of their introduction throughout the lifecycle of the code. I also want to be able to write contracts that overlap with programmatic checks in my code, deliberately or accidentally. In order to do so, I must be able to provide my users with builds where the contracts aren't assumed, and assumptions need to be enabled separately.

I can do that with C++20 if I really need to do so in anger; it requires macroing all contracts, and making them invisible to the compiler after preprocessing when it's necessary to switch contract checking off and assumption also off. This is going to be the default setup that I ship.

The downsides of such approaches are fairly obvious: there's reliance on the preprocessor, the approach requires a library-specific macro, and if this approach proves popular, we will have a proliferation of such macros, and it's going to be quite a scaling challenge to enable contracts in any large program. Note that that applies even for

cases where no assumption is done - to enable `*checking*`, you're going to need to define multiple macros to convince various libraries to conditionally compile the contract attributes into the source code.

With the assumption mode, I do not need to pollute my source code with such nonsense. I can make a recommendation to run with assumptions off first, and switch assumptions on after sufficient testing has been performed to increase confidence that it's the right thing to do and that the time for it is right. With an assumption mode, I can use C++20 contracts the way they were intended to be used; the same source code can be mapped to multiple different use cases, without having to resort to macros. All this proposal is doing is adding one more mapping target, and arguably an important one that is currently missing.

Without the assumption mode, things become much more awkward. The `checking=off` mode is what everybody will eventually want to run at some point, some users want to run that mode most of the time, and if they can, all the time, regardless of what contracts buy them as a debugging aid. Even small amounts of fairly lightweight checking are going to be turned off in many "release"/"production" scenarios. But not all those scenarios are such that they also work with assumptions enabled.

Wording

In [basic.def.odr]/12.6, modify as follows:

if D invokes a function with a precondition, or is a function that contains an assertion or has a contract condition (9.11.4), it is implementation-defined under which conditions all definitions of D shall be translated using the same build level and violation continuation mode and contract assumption mode; and

In [dcl.attr.contract.check]/3, modify as follows:

The translation of a program consisting of translation units where the build level, continuation mode, and contract assumption mode ~~is~~ are not the same in all translation units is conditionally-supported.

In [dcl.attr.contract.check]/4, modify as follows:

A translation may be performed with one of the following contract assumption modes: off or on.
If no contract assumption mode is explicitly selected, the default contract assumption mode is off.
During constant expression evaluation (7.7), only predicates of checked contracts are evaluated. In other contexts, it is unspecified whether the predicate for a contract that is not checked under the current build level is evaluated; if the predicate of such a contract would evaluate to false and contract assumption mode is on, the behavior is undefined.